

# Introducing sorting algorithms

In this session, we'll introduce algorithms for the problem of sorting, or re-arranging the elements of a collection in order. We'll consider two simple  $O(n^2)$  algorithms in detail:

- Selection sort
- Insertion sort

You will need to be able to demonstrate the operation of these algorithms on sample inputs, and also to describe their properties.

In week 7 we'll consider more efficient approaches. However the simple approaches may be preferred for small inputs.

## Revision: asymptotic complexity (big-O)

Working out the order of a sum of different powers of  $n$  is simply a matter of discarding all but the highest power of  $n$ , and ignoring the constant in front:

- 14 is  $O(1)$ : constant
- $3n^2 + 26$  is  $O(n^2)$ : quadratic
- $n^3 + 4n^2$  is  $O(n^3)$ : cubic

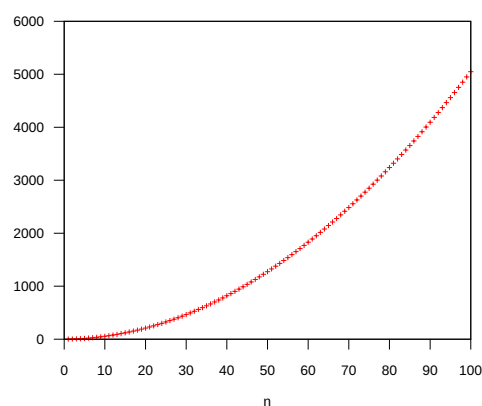
However this function is more tricky:

$$n + (n-1) + (n-2) + \dots + 2 + 1$$

As can be seen from the plot of this function at right, it isn't  $O(n)$ . If it were, all the points would be on a straight line.

There are several ways to work out this sum. One method is to look at it graphically, in terms of the area covered by blocks of length  $n$ ,  $n-1$ , etc. We can simplify the problem by putting the sum together with a copy, flipped vertically:

| $n$ | $n + (n-1) + (n-2) + \dots + 2 + 1$ |
|-----|-------------------------------------|
| 1   | 1                                   |
| 2   | 3                                   |
| 3   | 6                                   |
| 4   | 10                                  |
| 5   | 15                                  |
| 6   | 21                                  |



Plot of  $n + (n-1) + (n-2) + \dots + 2 + 1$

|     |  |     |   |
|-----|--|-----|---|
| n   |  |     | 1 |
| n-1 |  |     | 2 |
| n-2 |  |     | 3 |
| ... |  | ... |   |
| ... |  | ... |   |
| ... |  | ... |   |
| ... |  | ... |   |
| 3   |  | n-2 |   |
| 2   |  | n-1 |   |
| 1   |  | n   |   |

This block is  $n+1$  wide and  $n$  high, so it has an area of  $n*(n+1)$ , and our original sum is half that:

$$n + (n-1) + (n-2) + \dots + 2 + 1 = n*(n+1)/2 = n^2/2 + n/2$$

which is  $O(n^2)$ .

Alternatively, since we're only worrying about the order of growth, we could observe that there are  $n$  numbers, none larger than  $n$ , so

$$n + (n-1) + (n-2) + \dots + 2 + 1 \leq n^2$$

Also, we know that half of them are at least  $n/2$ , so

$$n^2/4 \leq n + (n-1) + (n-2) + \dots + 2 + 1$$

so the sum lies between two functions that are both  $O(n^2)$ , so it is also  $O(n^2)$ .

## Sorting

**A**

- accordion, layouts
  - about 125
  - move form, adding 131
  - nothing, in tab 128, 129
  - toolbar, adding 129-131
- adapters, Ext
  - about 15
  - using 15, 20
- Adobe AIR 285
- Adobe Integrated Run time. *See* Adobe AIR
- AJAX 12
- Asynchronous JavaScript and XML. *See* AJAX

**B**

- built-in features, Ext
  - client-side sorting 86
  - columns, reordering 86, 87
  - columns, hidden 86
  - columns, visible 86
- button, toolbars
  - creating 63
  - handlers 67, 68
  - icon buttons 67
  - split button 64
  - buttons, form 53

**C**

- cell renderers
  - about 69
- lookup data stores, creating 83
- two columns, combining 94
- classes 254
- ComboBox, form
  - about 47
- database-driven 47-50
- component config 59
- config object
  - about 25, 29
  - new way 25, 29
  - old way 25
  - tips 25, 29
- content, loading on menu item click 68, 69
- custom class, creating 256-259
- custom component, creating 264-266
- custom events, creating 262-264

**D**

- data, filtering
  - about 238
  - remote, filtering 238-244
- data, finding
  - about 237
  - by field value 237
  - by record ID 238
  - by record index 237
- data, formatting
  - about 278
  - data, formatting 279
  - other formatting 280, 281
  - string, formatting 278
- data, displaying, GridPanel

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
| a | 27 | 14 | 77 | 85 | 34 | 16 | 58 |

we want to re-arrange the elements to

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
| a | 14 | 16 | 27 | 34 | 58 | 77 | 85 |

Formally, we want a re-arrangement (a *permutation* in mathematical terminology) of the original elements that is *ordered*:

$$a[0] \leq a[1] \leq a[2] \leq \dots \leq a[n-2] \leq a[n-1]$$

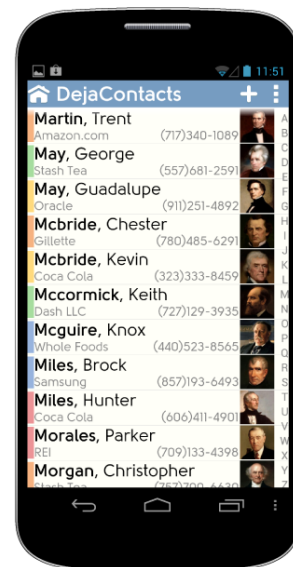
Sorting is a fundamental computing task.

- We may sort data for presentation to users, e.g. phone books, directory listings, indexes of books.
- Sorting is an important building-block in larger algorithms, because it simplifies tasks such as searching for a particular key or detecting duplicates.

Sort keys can be numbers, strings, anything that has an ordering. We'll use numbers in our examples but be careful to distinguish an array index **i** from the value **a[i]** at that location.

## Evaluating sorting algorithms

- How much space does the algorithm require, in addition to the original array? At best this will be  $O(1)$ , but some algorithms achieve faster times by using more memory (space-time trade-off).



- How much time does it take? We shall count comparison operations, and consider both average and worst case.
- Does the algorithm perform better on sorted inputs?

An adaptive sort is one that performs better on partially sorted inputs, and takes  $O(n)$  time on sorted or almost-sorted inputs.

This can be useful in some applications.

- What happens if there are repeated keys?

A stable sort is one that preserves the relative ordering of elements that compare as equal.

This is a useful property, e.g. we can sort people by first name, and then sort by surname, and get them in surname, first name order.

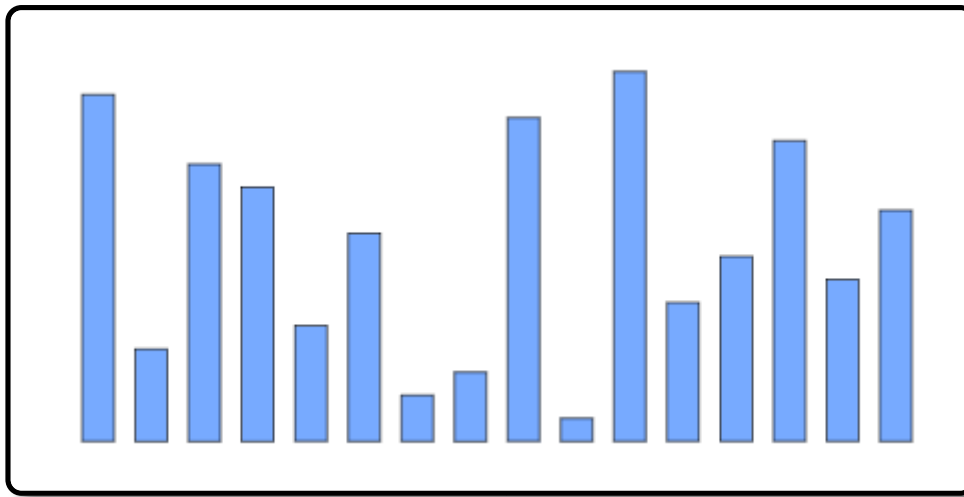
We can make any algorithm stable by including the original position in the sort, but this costs  $O(n)$  space and extra time.

## Selection sort

The idea of selection sort is to build up a collection of the smallest elements, in order, at the start of the array. To add a new element to this collection, find the smallest in what's left, and swap it with the first element after our position.

To get a feel for the process, try to apply the above strategy as mechanically as you can to sort the array below using the applet below. This is a modification of the applet from week 1, with the initial subarray of least elements in grey. You will always be swapping with the next element.

- Click on the least element in the remaining part to swap it with the first element in that part (which may be the same element). The collection of smallest elements on the left will then grow by one.
- Pressing **Reset** re-shuffles the array.



Notice that after we select the second-last element, there is no need to select the last one, as it must be the biggest.

Now let's recast that as a high-level description of the algorithm, leaving the detail of how we find the smallest remaining element till later:

Algorithm SelectionSort( $a[0..n-1]$ )

$i \leftarrow 0$

invariant:  $a[0..i-1]$  contains the  $i$  smallest elements in ascending order

WHILE  $i < n-1$

$\text{min} \leftarrow \text{index of the smallest element in } a[i..n-1]$

    Swap( $a, i, \text{min}$ )

$i \leftarrow i+1$

On termination of the loop, we have  $i = n-1$ , so  $a[0..n-2]$  contains the  $n-1$  smallest elements in ascending order. Therefore the remaining element  $a[n-1]$  must be at least as big as those, so the whole array is in order.

## Example

Let's work through the operation of this algorithm on a sample array. We consider the state of the array between iterations of the outer loop.

i = 0

|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
|---|----|----|----|----|----|----|----|
| a | 27 | 14 | 77 | 85 | 34 | 16 | 58 |

The smallest element is 14, at position 1, so we need to swap that to position 0.

i = 1

|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
|---|----|----|----|----|----|----|----|
| a | 14 | 27 | 77 | 85 | 34 | 16 | 58 |

The next smallest element is 16, at position 5, so we need to swap that to position 1.

i = 2

|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
|---|----|----|----|----|----|----|----|
| a | 14 | 16 | 77 | 85 | 34 | 27 | 58 |

The next smallest element is 27, which we just moved to position 5. Now we need to swap it down to position 2.

i = 3

|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
|---|----|----|----|----|----|----|----|
| a | 14 | 16 | 27 | 85 | 34 | 77 | 58 |

The next smallest element is 34, which is at position 4. We need to swap it down to position 3.

i = 4

|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
|---|----|----|----|----|----|----|----|
| a | 14 | 16 | 27 | 34 | 85 | 77 | 58 |

The next smallest element is 58, which is at position 6. We need to swap it down to position 4.

i = 5

|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
|---|----|----|----|----|----|----|----|
| a | 14 | 16 | 27 | 34 | 58 | 77 | 85 |

The next smallest element is 77, which is already at position 5, so we leave it there.

i = 6

|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
|---|----|----|----|----|----|----|----|
| a | 14 | 16 | 27 | 34 | 58 | 77 | 85 |

We have reached n-1, so we exit the loop. The last remaining element must be at least as large as those we've collected, so the whole array is now ordered.

|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
|---|----|----|----|----|----|----|----|
| a | 14 | 16 | 27 | 34 | 58 | 77 | 85 |

## Finding the smallest element

To complete the algorithm, we need to spell out the line

```
min ← index of the smallest element in a[i..n-1]
```

This is similar to the maximum exercise from week 1:

```
min ← i
j ← i+1
invariant: min is the index of the smallest
element in a[i..j-1]
WHILE j < n
    IF a[j] < a[min]
        min ← j
    j ← j+1
```

When we reach the end of the loop  $j = n$ , so the invariant tells us that **min** is the index of the smallest element in  $a[i..n-1]$ , and this is the position we want to swap with **i**. It could be that  $\text{min} = i$ , in which case the swap is unnecessary but harmless.

Plugging this into the top-level loop gives the complete algorithm:

Algorithm SelectionSort( $a[0..n-1]$ )

```
i ← 0
invariant: a[0..i-1] contains the i smallest elements in ascending order
WHILE i < n-1
    min ← i
    j ← i+1
    invariant: min is the index of the smallest element in a[i..j-1]
    WHILE j < n
        IF a[j] < a[min]
            min ← j
        j ← j+1
    Swap(a, i, min)
```

$i \leftarrow i+1$

Here  $i$  is the control variable for the outer loop, while  $j$  is the control variable for the inner one.

Step through the working of selection sort:

| n = 7 | 0                    | 1  | 2  | 3  | 4  | 5  | 6  |
|-------|----------------------|----|----|----|----|----|----|
| a     | 27                   | 14 | 77 | 85 | 34 | 16 | 58 |
| i     | <input type="text"/> |    |    |    |    |    |    |
| min   | <input type="text"/> |    |    |    |    |    |    |
| j     | <input type="text"/> |    |    |    |    |    |    |

**SelectionSort(a[0..n-1])**  
 $i \leftarrow 0$   
WHILE  $i < n-1$   
     $min \leftarrow i$   
     $j \leftarrow i+1$   
    WHILE  $j < n$   
        IF  $a[j] < a[min]$   
             $min \leftarrow j$   
     $j \leftarrow j+1$   
    Swap(a, i, min)  
     $i \leftarrow i+1$

Reset Single step Big step Run Stop

## Evaluation of selection sort

Selection sort uses  $O(1)$  extra space, for several temporary variables.

The time complexity can be seen from the structure of the loops, as shown at right. Whatever data is in the array, selection sort does  $(n-1) + (n-2) + \dots + 3 + 2 + 1$  comparisons. We know from an earlier calculation that this is  $n*(n-1)/2$ , which is  $O(n^2)$ .

This also means that selection sort is not an adaptive sort.

The swap operation can change the order of equal elements, so selection sort is not a stable sort.

Algorithm  
SelectionSort(a[0..n-1])

```
 $i \leftarrow 0$ 
WHILE  $i < n-1$ 
    ...
     $j \leftarrow i+1$ 
    WHILE  $j < n$ 
        ... comparison ...
         $j \leftarrow j+1$ 
    ...
     $i \leftarrow i+1$ 
```

There are some obvious inefficiencies in selection sort, which point the way to better algorithms:

- The swap operation moves a small element down, but it moves a more-or-less random element



up. Sometimes it moves a fairly small element up, and later moves it down again. The quicksort algorithm (week 7) also uses swaps over long distances, but it always moves a large element up at the same time as it moves a small one down.

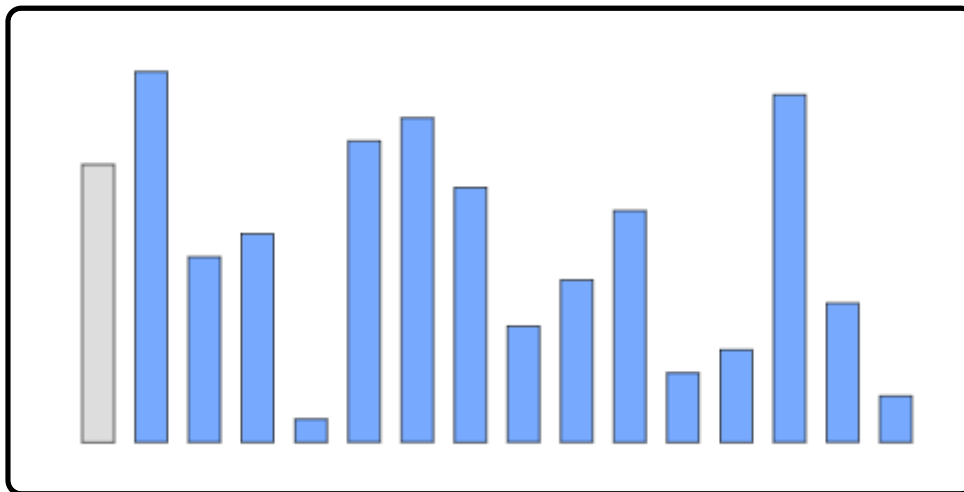
- Selection sort has to search through the whole remainder of the array to find the least element. Heapsort speeds this up with a clever organization of the rest of the array called a *heap* (which you'll meet next year in Data Structures and Algorithms).

## Insertion sort

Insertion sort operates by building an ordered sub-array on the left, expanding it by inserting each element from the unsorted portion into its appropriate place in the ordered sub-array.

To get a feel for the process, try to apply the above strategy as mechanically as you can to sort the array below using the applet below. This is also a modification of the applet from week 1, with the initial subarray of ordered elements in grey.

- At each stage, you must indicate (by clicking on a position) where the next element should be moved to in order to insert it into the sorted portion of the array. This may be one of the positions to its left, or its current position. Inserting the element will entail moving the intervening elements to make room for the element being moved.
- Pressing **Reset** re-shuffles the array.



Note that the process starts with the first element by itself already ordered, so the first element we have to insert is the second one in the array.

Here's a high-level description of this strategy, leaving the detail of how we do the insertion till later:

## Algorithm InsertionSort( $a[0..n-1]$ )

$i \leftarrow 1$

invariant:  $a[0..i-1]$  in ascending order

**WHILE**  $i < n$

    insert  $a[i]$  in place in  $a[0..i-1]$

$i \leftarrow i+1$

Notice that we start with  $i = 1$ , because the sub-array consisting of just the first element  $a[0]$  is trivially ordered. On termination of the loop,  $i = n$ , so  $a[0..n-1]$  is in order.

## Example

Let's use this algorithm to sort this array:

|       |    |    |    |    |    |    |    |                                                                                                     |
|-------|----|----|----|----|----|----|----|-----------------------------------------------------------------------------------------------------|
|       | 0  | 1  | 2  | 3  | 4  | 5  | 6  |                                                                                                     |
| a     | 27 | 14 | 77 | 85 | 34 | 16 | 58 | The sub-array consisting of the first element is trivially ordered.                                 |
| i = 1 | 0  | 1  | 2  | 3  | 4  | 5  | 6  |                                                                                                     |
| a     | 27 | 14 | 77 | 85 | 34 | 16 | 58 | To extend the ordered sub-array, we must insert 14, by moving 27 to the right.                      |
| i = 2 | 0  | 1  | 2  | 3  | 4  | 5  | 6  |                                                                                                     |
| a     | 14 | 27 | 77 | 85 | 34 | 16 | 58 | The next number 77 is already larger than the numbers so far, so we can leave it where it is.       |
| i = 3 | 0  | 1  | 2  | 3  | 4  | 5  | 6  |                                                                                                     |
| a     | 14 | 27 | 77 | 85 | 34 | 16 | 58 | Again the next number 85 is already larger than the numbers so far, so we can leave it where it is. |
| i = 4 | 0  | 1  | 2  | 3  | 4  | 5  | 6  |                                                                                                     |
| a     | 14 | 27 | 77 | 85 | 34 | 16 | 58 | To extend the ordered sub-array, we must insert 34, by moving 77 and 85 to the right.               |
| i = 5 | 0  | 1  | 2  | 3  | 4  | 5  | 6  |                                                                                                     |
| a     | 14 | 27 | 34 | 77 | 85 | 16 | 58 | To extend the ordered sub-array, we must insert 16, by moving four numbers to the right.            |

i = 6

|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
|---|----|----|----|----|----|----|----|
| a | 14 | 16 | 27 | 34 | 77 | 85 | 58 |

To extend the ordered sub-array, we must insert 58, by moving 77 and 85 to the right.

i = 7

|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
|---|----|----|----|----|----|----|----|
| a | 14 | 16 | 27 | 34 | 58 | 77 | 85 |

We have reached the end of the array, and now the whole array is ordered.

## Insertion

To complete the algorithm, we need to flesh out the step

insert  $a[i]$  in place in  $a[0..i-1]$

The idea is to search backward from position  $i$  for the right position. As we search, we move the elements greater than  $a[i]$  that we encounter one place to the right. Then when we find the place where  $a[i]$  belongs, we will have a slot to put it in.

Consider the situation when  $i = 4$ :

|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
|---|----|----|----|----|----|----|----|
| a | 14 | 27 | 77 | 85 | 34 | 16 | 58 |

We want to move the elements of the sorted sub-array that are larger than  $a[i] = 34$ , namely 77 and 85, to the right so we can slot in the value 34. To do this, we need to save  $a[i]$  in a temporary variable:

j = 4

|     | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
|-----|----|----|----|----|----|----|----|
| tmp | 34 |    |    |    |    |    |    |
| a   | 14 | 27 | 77 | 85 | 34 | 16 | 58 |

85 is larger than 34, so copy it to the right.

j = 3

|     | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
|-----|----|----|----|----|----|----|----|
| tmp | 34 |    |    |    |    |    |    |
| a   | 14 | 27 | 77 | 85 | 85 | 16 | 58 |

Similarly, 77 is larger than 34, so copy it to the right.

j = 2

|     | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
|-----|----|----|----|----|----|----|----|
| tmp | 34 |    |    |    |    |    |    |
| a   | 14 | 27 | 77 | 77 | 85 | 16 | 58 |

The next element 27 isn't larger than 34, so exit the loop and copy **tmp** to

$a[j]$ .

|     |    |   |    |    |    |    |    |    |    |
|-----|----|---|----|----|----|----|----|----|----|
|     |    |   | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
| tmp | 34 | a | 14 | 27 | 34 | 77 | 85 | 16 | 58 |

Here is the pseudocode to do the insertion:

```
tmp ← a[i]
j ← i
invariant: a[0..j-1], a[j+1..i] in ascending order and tmp < a[j+1..i]
WHILE j > 0 AND a[j-1] > tmp
    a[j] ← a[j-1]
    j ← j-1
a[j] ← tmp
```

At the end of the loop, all the positions  $a[j+1..i]$  are larger than **tmp**, and either

- $j = 0$ , so copying **tmp** to  $a[0]$  leaves  $a[0..i]$  in order, or
- all the elements  $a[0..j-1]$  are less than or equal to **tmp**, so copying **tmp** to  $a[j]$  leaves  $a[0..i]$  in order.

Plugging this in gives the complete insertion sort algorithm:

Algorithm InsertionSort( $a[0..n-1]$ )

```
i ← 1
invariant: a[0..i-1] in ascending order
WHILE i < n
    tmp ← a[i]
    j ← i
    invariant: a[0..j-1], a[j+1..i] in ascending order and tmp < a[j+1..i]
    WHILE j > 0 AND a[j-1] > tmp
        a[j] ← a[j-1]
        j ← j-1
    a[j] ← tmp
    i ← i+1
```

Step through the working of insertion sort:

| n = 7 | 0                    | 1  | 2  | 3  | 4  | 5  | 6  |
|-------|----------------------|----|----|----|----|----|----|
| a     | 27                   | 14 | 77 | 85 | 34 | 16 | 58 |
| i     | <input type="text"/> |    |    |    |    |    |    |
| tmp   | <input type="text"/> |    |    |    |    |    |    |
| j     | <input type="text"/> |    |    |    |    |    |    |

**InsertionSort(a[0..n-1])**  
  
**i** ← 1  
**WHILE** i < n  
    tmp ← a[i]  
    j ← i  
    **WHILE** j > 0 AND a[j-1] > tmp  
        a[j] ← a[j-1]  
        j ← j-1  
    a[j] ← tmp  
    i ← i+1

## Evaluation of insertion sort

Insertion sort uses  $O(1)$  extra space, for several temporary variables.

Again we can work out the time complexity by examining the loops. In the worst case, this algorithm does  $1 + 2 + 3 + \dots + (n-2) + (n-1)$  comparisons, which we know is  $O(n^2)$ .

However, the inner loop of this algorithm has an extra condition, which depends on the input data. In particular, if **a[i]** is at least as big as the preceding elements, the inner loop exits after only one comparison. If the initial array is already ordered, the algorithm does only  $n-1$  comparisons, which is  $O(n)$ . Thus insertion sort is an adaptive sort.

The same test ensures that we never move **a[i]** past an equal element. Thus insertion sort is also a stable sort.

## Can we do better?

One idea for a faster sort is to move things over longer distances by first sorting the even and odd positions separately using a simple adaptive sort, such as insertion sort. So if we start with the array

Algorithm  
InsertionSort(a[0..n-1])

```
i ← 1
WHILE i < n
    tmp ← a[i]
    j ← i
    WHILE j > 0 AND a[j]
        a[j] ← a[j-1]
        j ← j-1
    a[j] ← tmp
    i ← i+1
```

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
| a | 27 | 14 | 77 | 85 | 34 | 16 | 58 |

Sorting the even positions yields

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
| a | 27 | 14 | 34 | 85 | 58 | 16 | 77 |

Sorting the odd positions yields

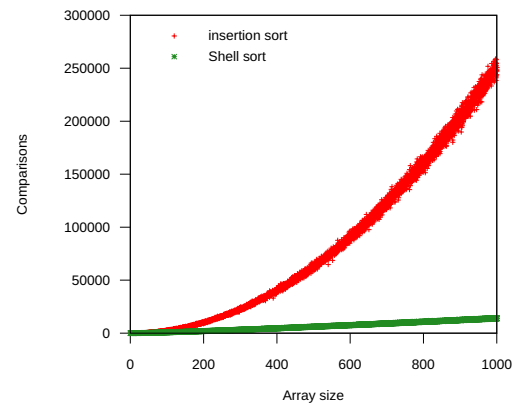
|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
| a | 27 | 14 | 34 | 16 | 58 | 85 | 77 |

We still need to sort the whole array, but now the array is nearly ordered, so our (adaptive) insertion sort will go faster:

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
| a | 14 | 16 | 27 | 34 | 58 | 77 | 85 |

Extending this idea with a whole sequence of intervals yields Shell sort (invented by Donald Shell in 1959). Shell sort is still only partially understood, but is a lot faster than the  $O(n^2)$  sorts. We'll be looking at even faster (and better-understood) sorts in week 7.

Moral: Tweaking your code yields only small improvements in performance, at the cost of making it harder to understand (and possibly wrong). Switching to a different algorithm can make your program enormously faster.



*Experimentally measured performance of Shell sort, using the gap sequence 1, 2, 4, 9, 20, 45, 101, 227, ... (good, but not the best known)*

Faster sorts exploit the fact that if we know that  $x \leq y$  and also  $y \leq z$ , then we also know that  $x \leq z$  without having to perform another comparison. These sorts use positioning in the array to remember the results of earlier comparisons, to save work later.